

Graph Coverage for Source Code

Draw the control flow graph for the **patternIndex()** method and define the appropriate test cases.

```
public class PatternIndex
{
    public static void main (String[] argv)
    {
        public static void main (String[] argv)
        {
            if (argv.length != 2)
            {
                System.out.println
                    ("java PatternIndex Subject Pattern");
                return;
            }
            String subject = argv[0];
            String pattern = argv[1];
            int n = 0;
            if ((n = patternIndex (subject, pattern)) == -1)
                System.out.println
                    ("Pattern string is not a substring of the subject string");
            else
                System.out.println
                    ("Pattern string begins at character " + n);
        }
    }

    /**
     * Find index of pattern in subject string
     *
     * @param subject String to search
     * @param pattern String to find
     * @return index (zero-based) of first occurrence of pattern in subject; -1 if
not found
     * @throws NullPointerException if subject or pattern is null
     */
    public static int patternIndex (String subject, String pattern)
    {
        final int NOTFOUND = -1;
        int iSub = 0, rtnIndex = NOTFOUND;
        boolean isPat = false;
        int subjectLen = subject.length();
        int patternLen = pattern.length();

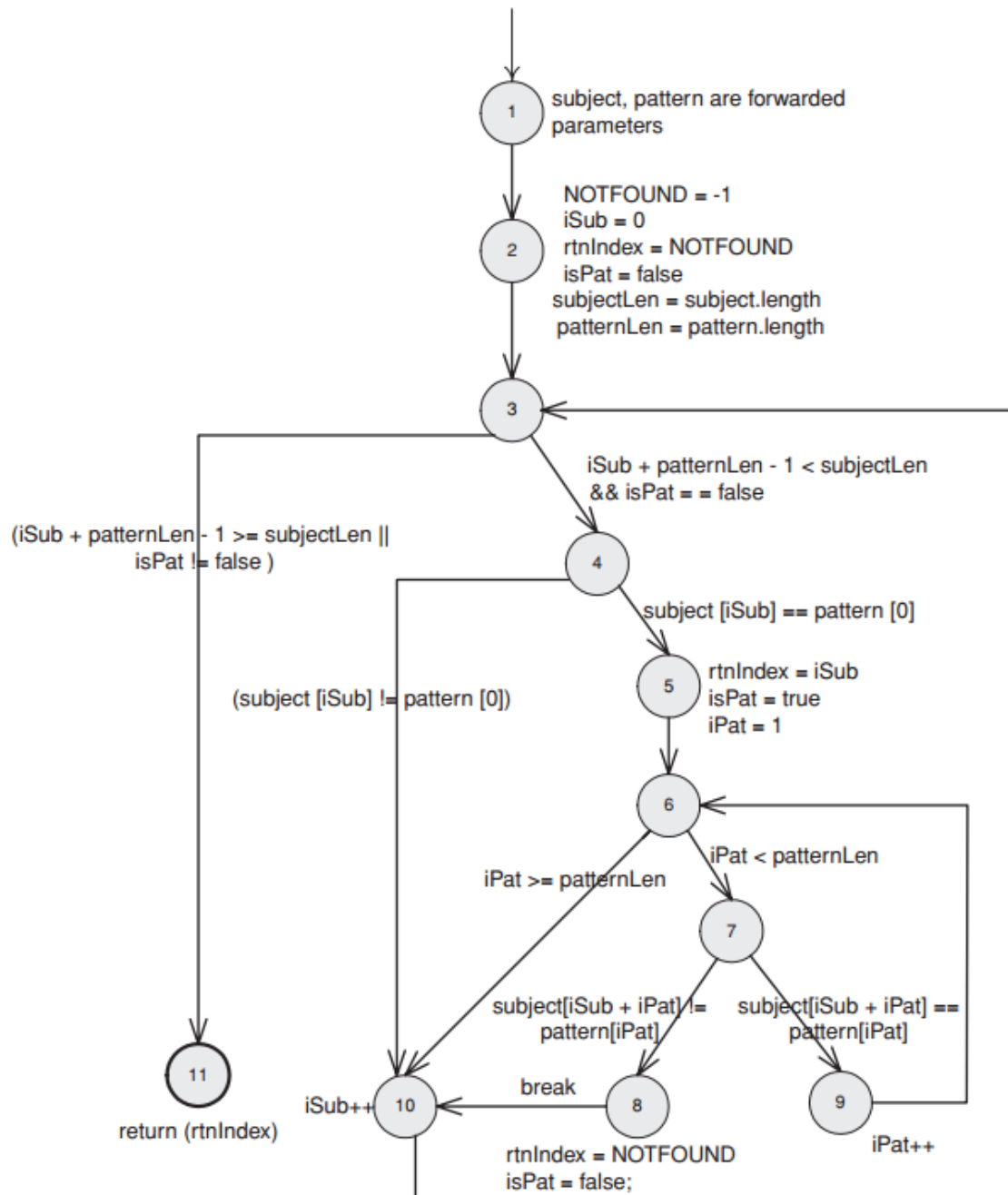
        while (isPat == false && iSub + patternLen - 1 < subjectLen)
        {
            if (subject.charAt(iSub) == pattern.charAt(0))
            {
                rtnIndex = iSub; // Starting at zero
                isPat = true;
                for (int iPat = 1; iPat < patternLen; iPat++)
                {
                    if (subject.charAt(iSub + iPat) != pattern.charAt(iPat))
                    {
                        rtnIndex = NOTFOUND;
                        isPat = false;
                        break; // out of for loop
                    }
                }
            }
            iSub++;
        }
        return rtnIndex;
    }
}
```

```

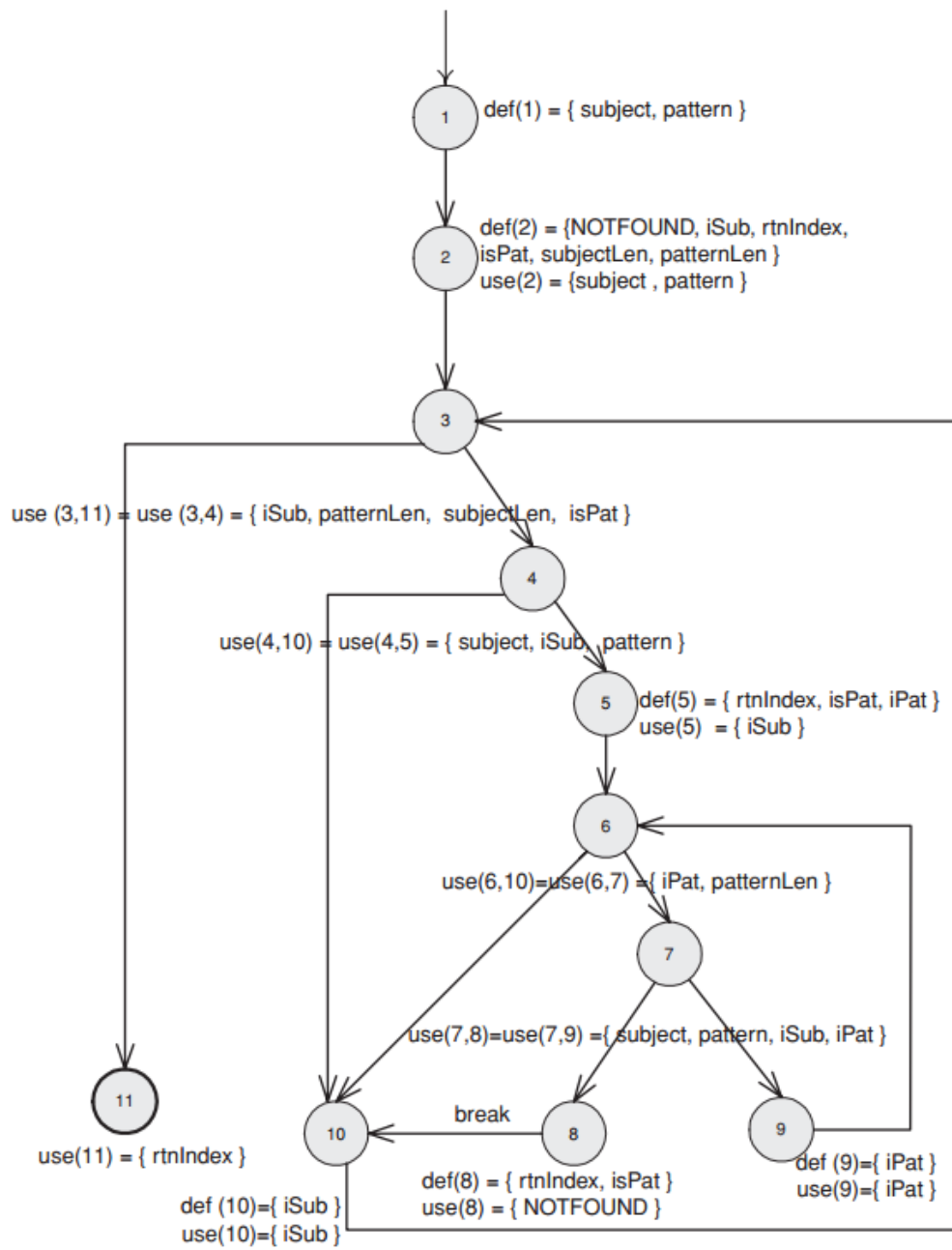
    }
    }
    iSub ++;
  }
  return (rtnIndex);
}

```

The CFG is defined below:



The graph with marked def and use of the variables is given below:



According to the defined CFG, the table below contains all defs and uses for each node separately:

Table 2.1. Defs and uses at each node in the CFG for TestPat		
node	def	use
1	{subject, pattern}	
2	{NOTFOUND, isPat, iSub, rtnIndex, subjectLen, patternLen}	{subject, pattern}
3		
4		
5	{rtnIndex, isPat, iPat}	{iSub}
6		
7		
8	{rtnIndex, isPat}	{NOTFOUND}
9	{iPat}	{iPat}
10	{iSub}	{iSub}
11		{rtnIndex}

According to the defined CFG, the table below contains all defs and uses for each edge separately:

Table 7.2. Defs and uses at each edge in the CFG for patternIndex().	
edge	use
(1, 2)	
(2, 3)	
(3, 4)	{iSub, patternLen, subjectLen, isPat}
(3, 11)	{iSub, patternLen, subjectLen, isPat}
(4, 5)	{subject, iSub, pattern}
(4, 10)	{subject, iSub, pattern}
(5, 6)	
(6, 7)	{iPat, patternLen}
(6, 10)	{iPat, patternLen}
(7, 8)	{subject, iSub, iPat, pattern}
(7, 9)	{subject, iSub, iPat, pattern}
(8, 10)	
(9, 6)	
(10, 3)	

Table 2.3 presents all du-paths for each def and use that are given in the tables above. The prefix “Yes” means that the specific du-path is already a subpath of some path for the same variable.

Table 2.3. Du-path sets for each variable in TestPat			
variable	du-path set	du-paths	prefix?
NOTFOUND rtnIndex	du (2, NOTFOUND)	[2,3,4,5,6,7,8]	
	du (2, rtnIndex)	[2,3,11]	
	du (5, rtnIndex)	[5,6,10,3,11]	
	du (8, rtnIndex)	[8,10,3,11]	
iSub	du (2, iSub)	[2,3,4]	Yes
		[2,3,4,5]	Yes
		[2,3,4,5,6,7,8]	Yes
		[2,3,4,5,6,7,9]	
		[2,3,4,5,6,10]	
		[2,3,4,5,6,7,8,10]	
		[2,3,4,10]	
		[2,3,11]	
	du (10, iSub)	[10,3,4]	Yes
		[10,3,4,5]	Yes
		[10,3,4,5,6,7,8]	Yes
		[10,3,4,5,6,7,9]	
		[10,3,4,5,6,10]	
		[10,3,4,5,6,7,8,10]	
		[10,3,4,10]	
		[10,3,11]	
iPat	du (5, iPat)	[5,6,7]	Yes
		[5,6,10]	
		[5,6,7,8]	
		[5,6,7,9]	
	du (9, iPat)	[9,6,7]	Yes
		[9,6,10]	
		[9,6,7,8]	
		[9,6,7,9]	
isPat	du (2, isPat)	[2,3,4]	
	du (5, isPat)	[2,3,11]	
		[5,6,10,3,4]	
		[5,6,10,3,11]	
	du (8, isPat)	[8,10,3,4]	
subject	du (1, subject)	[8,10,3,11]	
		[1,2]	Yes
		[1,2,3,4,5]	Yes
		[1,2,3,4,10]	
		[1,2,3,4,5,6,7,8]	
pattern	du (1, pattern)	[1,2,3,4,5,6,7,9]	
		[1,2]	Yes
		[1,2,3,4,5]	Yes
		[1,2,3,4,10]	
		[1,2,3,4,5,6,7,8]	
subjectLen	du (2, subjectLen)	[1,2,3,4,5,6,7,9]	
patternLen	du (2, patternLen)	[2,3,4]	
		[2,3,11]	
		[2,3,4]	Yes
		[2,3,11]	
		[2,3,4,5,6,7]	
		[2,3,4,5,6,10]	

The final test paths that cover all du-paths are given in Table 2.4.

Table 2.4. Test paths to satisfy all du-paths coverage on TestPat

test case (subject,pattern,output)	test path(t)
(a, bc, -1)	[1,2,3,11]
(ab, a, 0)	[1,2,3,4,5,6,10,3,11]
(ab, ab, 0)	[1,2,3,4,5,6,7,9,6,10,3,11]
(ab, ac, -1)	[1,2,3,4,5,6,7,8,10,3,11]
(ab, b, 1)	[1,2,3,4,10,3,4,5,6,10,3,11]
(ab, c, -1)	[1,2,3,4,10,3,4,10,3,11]
(abc, abc, 0)	[1,2,3,4,5,6,7,9,6,7,9,6,10,3,11]
(abc, abd, -1)	[1,2,3,4,5,6,7,9,6,7,8,10,3,11]
(abc, ac -1)	[1,2,3,4,5,6,7,8,10,3,4,10,3,11]
(abc, ba, -1)	[1,2,3,4,10,3,4,5,6,7,8,10,3,11]
(abc, bc, 1)	[1,2,3,4,10,3,4,5,6,7,9,6,10,3,11]

According to the found test paths, test cases are appropriately defined, and they are used to create parameterized tests. Below is the corresponding source code for generating JUnit parameterized tests for the PatternIndex code

```
// JUnit tests for PatternIndex.java
package org.example; // JUnit tests for PatternIndex.java
import org.junit.Test;
import org.junit.runner.RunWith;
import org.junit.runners.Parameterized;

import java.util.*;

import static org.junit.Assert.assertEquals;

@RunWith(Parameterized.class)
public class DataDrivenPatternIndexTest
{
    private final String subject;
    private final String pattern;
    private final int expected;

    public DataDrivenPatternIndexTest (String subject, String pattern, int
expected)
    {
        this.subject = subject;
        this.pattern = pattern;
        this.expected = expected;
    }

    @Parameterized.Parameters
    public static Collection<Object[]> parameters()
    {
        return Arrays.asList (new Object [][] {
            {"a", "bc", -1},
            {"ab", "a", 0},
            {"ab", "ab", 0},
        });
    }
}
```

```
    {"ab", "ac", -1},
    {"ab", "b", 1},
    {"ab", "c", -1},
    {"abc", "abc", 0},
    {"abc", "abd", -1},
    {"abc", "ac", -1},
    {"abc", "ba", -1},
    {"abc", "bc", 1}
  });
}
```

```
@Test
public void dataFlowTests()
{
    assertEquals ("Data flow PatternIndex test",
        PatternIndex.patternIndex (subject, pattern), expected);
}
}
```